

HW1_Statmodel

2025-02-07

##Problem 1

```
#install.packages('faraway')
library(faraway)
data(prostate)
head(prostate, 5)
```

```
##      lcavol lweight age      lbph svi      lcp gleason pgg45      lpsa
## 1 -0.5798185  2.7695  50 -1.386294  0 -1.38629      6      0 -0.43078
## 2 -0.9942523  3.3196  58 -1.386294  0 -1.38629      6      0 -0.16252
## 3 -0.5108256  2.6912  74 -1.386294  0 -1.38629      7     20 -0.16252
## 4 -1.2039728  3.2828  58 -1.386294  0 -1.38629      6      0 -0.16252
## 5  0.7514161  3.4324  62 -1.386294  0 -1.38629      6      0  0.37156
```

#Part a

```
modell1 = lm(formula = lpsa ~ ., data = prostate)
summary(modell1)
```

```
##
## Call:
## lm(formula = lpsa ~ ., data = prostate)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.7331 -0.3713 -0.0170  0.4141  1.6381
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.669337   1.296387   0.516  0.60693
## lcavol       0.587022   0.087920   6.677 2.11e-09 ***
## lweight      0.454467   0.170012   2.673  0.00896 **
## age         -0.019637   0.011173  -1.758  0.08229 .
## lbph         0.107054   0.058449   1.832  0.07040 .
## svi          0.766157   0.244309   3.136  0.00233 **
## lcp         -0.105474   0.091013  -1.159  0.24964
## gleason      0.045142   0.157465   0.287  0.77503
## pgg45        0.004525   0.004421   1.024  0.30886
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.7084 on 88 degrees of freedom
## Multiple R-squared:  0.6548, Adjusted R-squared:  0.6234
## F-statistic: 20.86 on 8 and 88 DF,  p-value: < 2.2e-16
confint(modell1, "lcavol", level = 0.95)

##           2.5 %    97.5 %
```

```
## lcavol 0.4122987 0.761745
```

#part b We use the bootstrap approach that resamples x's and y's in respective pairs.

```
#install.packages('faraway')
#install.packages('ggplot2')
library(faraway)
library(ggplot2)
data(prostate)
```

```
B <- 1000
```

```
beta1 <- numeric(B) #as we have 1000 bootstrap values
```

```
set.seed(123)
```

```
for (b in 1:B) {
```

```
  samp <- sample(1:nrow(prostate), nrow(prostate), replace=TRUE)
```

```
#we're creating a new bootstrap data by selecting the rows we got from samp
```

```
  bootstrap <- prostate[samp, ]
```

```
  boot_model <- lm(lpsa ~ ., data = bootstrap) #fitting the same model as part a on our bootstrap data
```

```
#as we're trying to find CI for lcavol we're taking coefficients for that from the bootstrap data
```

```
  beta1[b] <- as.numeric(coef(boot_model)["lcavol"]) }
```

```
#this creates a vector with 1000 values where ith element would be coeff of lcavol from our ith bootstrap
```

```
print(beta1[1:10])
```

```
## [1] 0.5455144 0.5513297 0.5559005 0.5663812 0.6154028 0.6418209 0.5438055
```

```
## [8] 0.7459284 0.5029057 0.5148310
```

Now, finding Confidence intervals

```
#standard error estimate
```

```
beta1_mean <- mean(beta1)
```

```
beta1_se <- sd(beta1)
```

```
print(c(beta1_mean - 1.96 * beta1_se, beta1_mean + 1.96 * beta1_se))
```

```
## [1] 0.4146043 0.7430557
```

```
#percentile
```

```
print(quantile(beta1, c(0.025, 0.975)))
```

```
##      2.5%      97.5%
```

```
## 0.4062082 0.7464578
```

```
#pivotal
```

```
model1 <- lm(lpsa ~ ., data = prostate)
```

```
beta1_hat <- as.numeric(coef(model1)["lcavol"])
```

```
upper <- as.numeric(quantile(beta1, 0.95, na.rm = TRUE))
```

```
lower <- as.numeric(quantile(beta1, 0.05, na.rm = TRUE))
```

```
print(c(2 * beta1_hat - upper, 2 * beta1_hat - lower))
```

```
## [1] 0.4606902 0.7430265
```

#partc Now using residuals and keeping x's fixed during bootstrap

```
library(faraway)
```

```
library(ggplot2)
```

```
data(prostate)
```

```
model <- lm(lpsa ~ ., data = prostate)
```

```
residuals <- model$residuals
```

```
fitted <- model$fitted.values
```

```
B <- 1000
```

```

beta1 <- numeric(B)
set.seed(123)
for (b in 1:B) {
  #we select 1000 residual values from length of it with replacement
  bootstrap_resi <- sample(residuals, length(residuals), replace=TRUE)
  #we're creating a new variable by adding fitted values of original to bootstrap residuals
  boot_y <- fitted + bootstrap_resi
  new_boot_data <- prostate
  #replacing the dependent variable y in original data with our new y calculated from fitted values and b
  new_boot_data$lpsa <- boot_y
  boot_modelr <- lm(lpsa ~ ., data = new_boot_data)
  beta1[b] <- as.numeric(coef(boot_modelr)["lcavol"])
}
head(new_boot_data,5)

```

```

##      lcavol lweight age      lbph svi      lcp gleason pgg45      lpsa
## 1 -0.5798185 2.7695 50 -1.386294 0 -1.38629      6      0 0.6033019
## 2 -0.9942523 3.3196 58 -1.386294 0 -1.38629      6      0 0.1324478
## 3 -0.5108256 2.6912 74 -1.386294 0 -1.38629      7     20 0.8704330
## 4 -1.2039728 3.2828 58 -1.386294 0 -1.38629      6      0 -0.9280304
## 5  0.7514161 3.4324 62 -1.386294 0 -1.38629      6      0 3.0111324

```

```
print(beta1[1:10])
```

```

## [1] 0.6189566 0.7309405 0.6015172 0.6760007 0.7349040 0.5112720 0.4831466
## [8] 0.6395562 0.6376369 0.6275009

```

Now, finding Confidence intervals

```

#standard error estimate
beta1_mean <- mean(beta1)
beta1_se <- sd(beta1)
print(c(beta1_mean - 1.96 * beta1_se, beta1_mean + 1.96 * beta1_se))

```

```
## [1] 0.4234326 0.7460537
```

```

#percentile
print(quantile(beta1, c(0.025, 0.975)))

```

```

##      2.5%      97.5%
## 0.4202447 0.7458902

```

```

#pivotal
model1 <- lm(lpsa ~ ., data = prostate)
beta1_hat <- as.numeric(coef(model1)["lcavol"])
upper <- as.numeric(quantile(beta1, 0.95, na.rm = TRUE))
lower <- as.numeric(quantile(beta1, 0.05, na.rm = TRUE))
print(c(2 * beta1_hat - upper, 2 * beta1_hat - lower))

```

```
## [1] 0.4526071 0.7302611
```

##Problem 2 #parta #Simulation starter code for Homework 1 ##Bootstrap for normal mean

```

nsims <- 5000
n <- 50
B <- 1000
#Matrix to store indicators for whether each interval
#covered the truth in each simulated dataset

```

```

cover <- matrix(nrow = nsims, ncol = 3)
set.seed(954567)
#Outer for-loop to repeatedly draw data
for(i in 1:nsims){
  #Draw data
  y <- rnorm(n)
  #Collect sample estimate
  mean_obs <- mean(y)
  #Initialize vector to store bootstrap samples
  boot_mean <- vector(length = B)
  #Inner for-loop for bootstrap
  for(b in 1:B){
    boot_y <- sample(y, replace = T)
    boot_mean[b] <- mean(boot_y)
  }
  #Bootstrap quantiles
  quants <- quantile(boot_mean, c(0.025, 0.975))
  #Save an indicator (true/false) for whether each interval contains the true mean (=0)
  cover[i,1]<- (0 > mean_obs*2-quants[2]) & (0 < mean_obs*2-quants[1])
  cover[i,2]<- (0 > quants[1]) & (0 < quants[2])
  cover[i,3]<- (0 > mean_obs - 1.96*sd(boot_mean)) & (0 < mean_obs + 1.96*sd(boot_mean))
}
#Examine the coverage rate across simulations for each interval method
colMeans(cover)

```

```
## [1] 0.939 0.942 0.942
```

First one is from pivotal method (0.939), second is percentile and third is standard error method for calculating confidence intervals. We can see that percentile and standard error have better coverage rate (both are equal in this case (0.942)) than pivotal method. Pivotal will be more accurate under general conditions than other two it would be better for skewed distribution, but in this case the data is normal and symmetric hence the standard error and percentile out-performed pivotal.

#partb

```

nsims <- 5000
n <- 50
B <- 1000
#Matrix to store indicators for whether each interval
#covered the truth in each simulated dataset
cover <- matrix(nrow = nsims, ncol = 3)
set.seed(954567)
#Outer for-loop to repeatedly draw data
for(i in 1:nsims){
  #Draw data
  y <- (rcauchy(n, location = 0, scale = 1)) #now for cuachy distribution
  #Collect sample estimate
  mean_obs <- mean(y)
  #Initialize vector to store bootstrap samples
  boot_mean <- vector(length = B)
  #Inner for-loop for bootstrap
  for(b in 1:B){
    boot_y <- sample(y, replace = T)
    boot_mean[b] <- mean(boot_y)
  }
}

```

```

#Bootstrap quantiles
quants <- quantile(boot_mean, c(0.025, 0.975))
#Save an indicator (true/false) for whether each interval contains the true mean (=0)
cover[i,1]<- (0 > mean_obs*2-quants[2]) & (0 < mean_obs*2-quants[1])
cover[i,2]<- (0 > quants[1]) & (0 < quants[2])
cover[i,3]<- (0 > mean_obs - 1.96*sd(boot_mean)) & (0 < mean_obs + 1.96*sd(boot_mean))
}
#Examine the coverage rate across simulations for each interval method
colMeans(cover)

```

```
## [1] 0.9834 0.9076 0.9752
```

In this case, Pivotal has the best coverage rate (0.984) followed by standard error(0.9752), and percentile has least coverage rate (0.9076). Cauchy distribution is similar to normal distribution but with heavier tails, it has more frequent extreme values. Pivotal performs the best as the condition for it is not as strong as the other two, making it best for cases like cauchy where there are heavy tails and it doesn't assume normal distribution making it better, the standard error method assumes normality as cauchy is similar to normal distribution it gives better coverage rate but gives larger confidence interval. Percentile method doesn't give best results as cauchy distribution is narrow, it uses percentiles from bootstrap without any correction leading to missing many outliers which are more frequent in cauchy.

#partc

```

nsims <- 5000
n <- 20
B <- 1000
#Matrix to store indicators for whether each interval
#covered the truth in each simulated dataset
cover <- matrix(nrow = nsims, ncol = 3)
set.seed(954567)
#Outer for-loop to repeatedly draw data
for(i in 1:nsims){
  #Draw data
  y <- (rcauchy(n, location = 0, scale = 1)) #now for cuachy distribution
  #Collect sample estimate
  mean_obs <- mean(y)
  #Initialize vector to store bootstrap samples
  boot_mean <- vector(length = B)
  #Inner for-loop for bootstrap
  for(b in 1:B){
    boot_y <- sample(y, replace = T)
    boot_mean[b] <- mean(boot_y)
  }
  #Bootstrap quantiles
  quants <- quantile(boot_mean, c(0.025, 0.975))
  #Save an indicator (true/false) for whether each interval contains the true mean (=0)
  cover[i,1]<- (0 > mean_obs*2-quants[2]) & (0 < mean_obs*2-quants[1])
  cover[i,2]<- (0 > quants[1]) & (0 < quants[2])
  cover[i,3]<- (0 > mean_obs - 1.96*sd(boot_mean)) & (0 < mean_obs + 1.96*sd(boot_mean))
}
#Examine the coverage rate across simulations for each interval method
colMeans(cover)

```

```
## [1] 0.9798 0.9004 0.9702
```

As we decreased sample size, confidence intervals are less reliable due to fewer data points. Pivotal still has

the best performance but coverage rate decreased compared to part b, this is due to reduced sample size due to which we cant estimate precise pivotal (eg, theta hat- theta). percentile depends only on bootstrap, as the sample size is small the CI becomes narrow which lead to drop in coverage rate hence its coverage dropped. In standard error, we use variance which fluctuates due to smaller sample size, leading to a drop in coverage rate.

#partd

```
nsims <- 5000
n <- 50
B <- 1000
#Matrix to store indicators for whether each interval
#covered the truth in each simulated dataset
cover <- matrix(nrow = nsims, ncol = 3)
set.seed(954567)
#Outer for-loop to repeatedly draw data
for(i in 1:nsims){
  #Draw data
  y <- runif(n, min = 0, max = 1)
  #Initialize vector to store bootstrap samples, here we use maximum value as unbiased estimator
  max_obs <- max(y)
  theta_hat <- max_obs * ((n + 1)/n)

  boot_max <- vector(length = B)
  #Inner for-loop for bootstrap
  for(b in 1:B){
    boot_y <- sample(y, replace = T)
    boot_max[b] <- max(boot_y) * ((n+1)/n)
  }
  #Bootstrap quantiles
  quants <- quantile(boot_max, c(0.025, 0.975))
  #Save an indicator (true/false) for whether each interval contains the true max (=1)
  cover[i,1]<- (1 > theta_hat*2-quants[2]) & (1 < theta_hat*2-quants[1])
  cover[i,2]<- (1 > quants[1]) & (1 < quants[2])
  cover[i,3]<- (1 > theta_hat - 1.96*sd(boot_max)) & (1< theta_hat + 1.96*sd(boot_max))
}
#Examine the coverage rate across simulations for each interval method
colMeans(cover)
```

```
## [1] 0.3350 0.6068 0.8886
```

Pivotal method method has the worst coverage rate (0.3350), the interval it gives contains the true max the least number of times, It uses (theta hat - theta) as pivot but still gives low coverage rate I am not sure why. percentile gives 0.6068 and standard error gives 0.8886 coverage rate.

##problem3 We are given a sample of size n from $U(0,1)$. $X_{(n)}$ is the maximum of the sample. We're selecting a bootstrap sample of size n.

#part a Each bootstrap sample has size n. Every number of original sample has equal chances of getting selected. So, probability that first bootstrap observation is $X_{(n)}$ is $\frac{1}{n}$.

#part b probability that we select $X_{(n)}$ in one draw is $\frac{1}{n}$ probability of not selecting $X_{(n)}$ in one draw is $1 - \frac{1}{n}$ as all draws are equally likely and independent, probability of not selecting $X_{(n)}$ in n draws is $(1 - \frac{1}{n})^n$ probability that $X_{(n)}$ is in the sample is, $1 - (1 - \frac{1}{n})^n$

#part c Need to show that, $P\left(X_{(n)}^{*(b)} = X_{(n)} \mid F_n\right) \rightarrow 1 - e^{-1}$, as $n \rightarrow \infty$ from part b, probability that $X_{(n)}$ is in the sample is, $1 - (1 - \frac{1}{n})^n$ As, $(1 - \frac{x}{n})^n \rightarrow e^{-x}$ as $n \rightarrow \infty$, $1 - (1 - \frac{1}{n})^n \rightarrow 1 - e^{-1}$ as $n \rightarrow \infty$

If $X_{(n)}$ is present in the bootstrap sample, that means it is in original sample, so $P\left(X_{(n)}^{*(b)} = X_{(n)} \mid F_n\right) \rightarrow 1 - e^{-1}$, as $n \rightarrow \infty$

#part d for $U(0,1)$, CDF is $F_Y(y) = P(Y < y) = y$, for $y \in [0, 1]$ as $X_{(n)}$ is maximum value of the sample, we can use $F_{Y_{(n)}}(y) = [F_Y(y)]^n$ hence $P(Y_{(n)} < y) = P(Y < y)^n$ so, $P(X_{(n)} < t) = t^n$, for $t \in [0, 1]$

#part e CDF of $n(1 - X_{(n)})$ is $P(n(1 - X_{(n)}) < t) = P(X_{(n)} > 1 - t/n) = 1 - P(X_{(n)} < 1 - t/n)$ from part d, CDF of $X_{(n)}$ is $P(X_{(n)} < t) = t^n$, for $t \in [0, 1]$ so, $1 - P(X_{(n)} < 1 - t/n) = 1 - (1 - \frac{t}{n})^n$ as $n \rightarrow \infty$, $\lim_{n \rightarrow \infty} (1 - \frac{t}{n})^n = e^{-t}$ so, CDF of $n(1 - X_{(n)})$ is $1 - e^{-t}$ which is CDF for exponential(1) random variable.

#part f